CHAPITRE 1 : Rappel BDD et SQL

I Rappel ORACLE :

SGBD ORACLE :

- né en 1979
- version dérivée d'un SGBD d'IBM : Système R

En plus des fonctionnalités classiques d'un SGBD Relationnel, Oracle offre :

a) des outils de développement d'application :
   1. générateur de tableur,
   2. gestionnaire de grille d'écran,
   3. générateur de rapport, etc. …

b) Des outils de gestion de BD repartis :
   1. accès à la BD sur des sites distants,
   2. possibilité d'avoir des données reparties sur plusieurs sites.

c) Outils d'administration, supervision et de cohérence de la BD

Oracle possède bien sûr les fonctionnalités classiques d'un SGBD Relationnel :

a) Définition et manipulation de données (LDD et LMD = SQL) :
   SQL est utilisable soit :
   - de façons interactives (SQLPlus)
   - a partir d'un langage hôte : C ; C++, fortran, prolog, etc.

b) La gestion de la cohérence de données

c) Gestion de la confidentialité
   1. possibilité d'avoir des vues (partie d'un BD qu'on attribue à un utilisateur)
   2. possibilité de donner des privilèges à un utilisateur ou à une classe d'utilisateurs

d) Gestion des contraintes d'intégrité : Il existe plusieurs types de contraintes :
   1. contraintes de type implicite
   2. contraintes de présence : « Null » ou « not Null »
   3. contraintes de référence
   4. etc.

e) Gestion des accès concurrents :
   1. techniques de verrouillage
   2. gestion de l'inter blocage.

II Le langage SQL (Structured Query Language)

II.1 Le LDD (Langage de Définition de Données)

Il existe 3 commandes :

- CREATE
- ALTER
- DROP

II.1.1 Commande de création de tables « CREATE »

II.1.1.1 Première forme de création de table

Syntaxe :

CREATE TABLE < *Nom_table*> (**définition_des_colonnes**) ;

Où « **définition_des_colonnes** » :

liste de : <*nom_col*> <type> [taille][Null / not Null]

Exemple1:

```
CREATE    TABLE       CLIENTS (
          nom         Char (20)      Not Null ,
          adresse     Char(80),
          val_compte  Number(6)) ;

CREATE    TABLE       CC (
          num_co      Number(6)      Not Null,
          nom         Char (20),
          marchandise Char(35),
          qte         Number(6),
          datecom     Date) ;

CREATE    TABLE       FOURNISSEUR (
          nom         Char (20),
          adresse     Char (80),
          marchandise Char (35),
          prix        Number (6)) ;
```

Attention : On utilisera cette base de données dans la suite du cours.

Rappel sur les types de données :

Les types les plus courants sont les suivants :

a) CHAINE DE CARACTERES

char(n), avec 1<=n<=255 :
Données de type chaîne de caractères de longueur fixe n

varchar2(n), avec 1<=n<=2000 :
Chaîne de caractères de longueur variable n

long :
Chaîne de caractère de longueur variable allant jusqu'à 2 Giga-Octets

Remarque : Les chaines de caractères et les dates (voir plus loin) se mettent toujours entre « quote » ' ', par exemple : ' BOUZIAD'.

b) NOMBRE

Number :
Réel compris entre -2^{37} et $+ 2^{37}$

Number(p,s) :
Nombre composé au maximum de *p* chiffres, dont *s* après la virgule

c) DATE Exemple : 01-JAN-98

Exemple 2: de création de table

Soit la table **Identité** simplifiée qui ne contient que le numéro d'identification, le nom, le prénom, la date de naissance, le nombre d'enfants, le revenu et l'adresse d'un individu. De plus :

1. Le numéro d'identité est un nombre entier
2. Le nom et le prénom d'une personne ne doit pas dépasser 30 caractères. Le type de ces colonnes sera donc un char(30) ou un varchar(30)
3. L'adresse peut être longue
4. le nombre d'enfants ne dépasse pas 2 chiffres, number (2)
5. Enfin pour le revenu nous ne nous intéresserons pas aux centimes.

La commande pour la création de cette table est la suivante (Sous Oracle) :

```
CREATE    TABLE     identite (
          ident                Number,
          nom                  varchar(30),
          prenom               varchar(30),
          date_naissance       date,
          nbr_enfants          number(2),
          revenu               number(12,0),
          adresse              varchar(200),
          ville                varchar(50),
          code                 postal Number (5),
          ) ;
```

II.1.1.2 Deuxième forme de création de table

Elle consiste à créer une table a l'aide d'une autre table (ou de plusieurs tables)

Syntaxe :

CREATE TABLE *<Nom_table>*
[(*Liste de noms de colonnes avec éventuellement Null ou Not Null*)]

AS < REQUÊTE > ;

Rq : une table peut donc être créer et alimenter à partir d'une requête.

Exemple :

```
CREATE    TABLE    Bons_Clients
AS

(SELECT   nom, adresse, val_compte        --  (ou *)
FROM      CLIENTS
WHERE     val_compte > 0) ;
```

II.1.2 Commande de modification de tables (ALTER)

II.1.2.1 Modification de la structure de tables

Elle consiste en l'ajout de colonnes.

Syntaxe :

```
ALTER    TABLE    <Nom_table>    ADD (définition_d'une_colonne) ;
```

Exemple :

```
ALTER    TABLE    Bons_Clients    ADD (tel Char(8)) ;
```

II.1.2.2 Modification du type des colonnes existantes

Syntaxe :

```
ALTER    TABLE    < Nom_table>    Modify  (définition des colonnes) ;
```

Dans la définition des colonnes, on peut redéfinir le type d'une colonne existante.

Exemple :

```
ALTER    TABLE    Bons_Clients    Modify (tel    char(10)) ;
```

II.1.3 Commande de suppression de tables (DROP) :

Syntaxe :

DROP TABLE < Nom_table> ;

Exemple :

DROP TABLE Bons _clients ;

Rq : La suppression d'une table supprime aussi tous ces synonymes (c'est un lien)

Rappel : synonyme

Syntaxes :

- CREATE SYNONYM <Nom_synonyme> FOR <Nom_table> ;
- DROP SYNONYM <Nom> ; -- supprime seulement le synonyme

II.2 Le Langage de Manipulation de Données (LMD)

Dans la suite de ce chapitre, les tableaux relatifs aux exemples sont à compléter par les étudiants.

II.2.1 Consultation (ou Requête Simples)

Syntaxe :

SELECT [ALL / DISTINCT] Liste_de_colonnes
FROM Liste_de_tables ;

Rq : pour sélectionner toutes les colonnes d'une table, l'instruction est : Select * FROM nom_table ;

Exemple 1 : Lister le nom et adresse de tous les clients.

Note : 'ALIAS'
Pour éviter de réécrire plusieurs fois les noms complet des tables dans les requêtes, on peut utiliser des 'alias', c'est à dire des abréviations des tables.

Exemple 2 : SELECT CLnom, CL.adresse
FROM CLIENTS CL ; -- *CL est un alias (abréviation) de la table CLIENTS*

II.2.2 Consultation avec qualifications/Conditions

Syntaxe :

SELECT [ALL / DISTINCT] Liste_de_colonnes
ROM Liste_de_tables
WHERE Conditions; -- *une condition est une Qualification*

Exemple : Lister toutes les commandes de ' BOUZIAD'

Attention : en SQL, il ya 3 familles de conditions ou de qualifications :

1. les conditions de comparaison,
2. les conditions de jointures,
3. les conditions de sous-requêtes.

II.2.2.1 Consultation avec conditions de comparaison

Quelques rappels sur les opérateurs qu'on peut utiliser :

1. Opérateurs de comparaisons :

Les opérateurs de comparaison reconnus par Oracle, sont les suivants

- =,>, <, < > ou != (pour differents), <= , >=

  Exemple :
  idclient > 2,
  idarticle = 6,
  raisonsociale = 'MOBILIS'

  Rq : Dans Oracle contrairement à MySQL, on peut faire des comparaisons entre couple ou entre n-uplets :

  (prixunit, stock) = (15000,40),
  (adresse, ville, codepost) = ('10 rue Hassiba Ben Bouali' , 'Rouiba' , '16008')

- Nous avons aussi respectivement les opérateurs :
  - appartenance à un intervalle :

    colonne [NOT] BETWEEN expression1 AND expression2

  - appartenance à une liste de valeurs :

    colonne [NOT] IN (liste de valeurs)

  - de similitude à une chaine de caractère :

    colonne [NOT] LIKE 'chaine'

  - d'égalité ou de différence avec l'élément nul :

    colonne IS [NOT] NULL

2. ***Opérateurs logiques liant les conditions :***

Il est possible de définir dans les conditions de la clause WHERE plusieurs conditions. Pour lier les conditions entre elles, on utilise les opérateurs logiques OR et AND.

Voici quelques exemples utilisant des conditions de comparaisons :

Exemples :

1. Lister les noms et adresses des fournisseurs, ou l'adresse existe et ou le nom commence par 'D' ou bien la 2éme lettre du nom est 'A' :

2. Sélectionner les dates de commandes comprise entre le 1er janvier 2009 et aujourd'hui :

3. Sélectionner les noms des clients dont les 2 premières lettres du nom sont 'DU', ‘PR’ ou 'MO' et n'importe quelle lettre pour les suivantes :

où *substr(chaine , m , n)* est une fonction qui extrait de la chaine *'chaine'* une chaine de longueur *n* à partir du m^{ième} caractère.

II.2.2.2 Consultation avec conditions de jointure

Il existe 4 types de jointures :

Définition 1 : Une ***jointure*** est un lien que l'on crée momentanément entre 2 tables disposant d'au moins une colonne commune.

Définition 2 : On appellera ***jointure simple*** une jointure qui met en relation une seule colonne de chacune des deux tables et une ***jointure multiple*** une jointure qui met en relation plusieurs colonnes de chacune des tables liées.

Définition 3 : Une ***auto jointure*** est une jointure entre une table et elle même.

Définition 4 : Une ***jointure externe*** sera une jointure simple dont le lien de jointure n'est respecté que par une seule des deux tables.

Exemple 1 : jointure simple

Nom et adresse des clients qui ont commandés des pommes ?

Exemple 2 : jointure multiple

Lister les nom et adresse des clients ayant commandes une marchandise fournie par 'BOUZIAD' ?

Rq : Les relations dans un SGBD tel qu'Oracle et MySql ne sont pas de vraies relations car il y a des duplications (doublons). Pour avoir une véritable relation (pas de doublons) alors utiliser :

SELECT **DISTINCT** …

Exemple 3 : Auto – jointure

Soit la table suivante :

EMPLOYE (IdEmp, Nom, Prénom, DateNaiss, Adresse, NomServ, chef)

Où chef est un IdEmp.

Question : on veut connaître le nom du chef de chaque employé ?

Exemple 4 : Jointure externe

- Une jointure externe est une jointure qui favorise une table par rapport à une autre. Les lignes de la table seront affiches même si la condition n'est pas réalisée.

- Pour distinguer la table favorisée (ou dominante) par rapport a l'autre qui est dominée il faut mettre un (+) du cote de la table dominée dans la jointure

Question : lister les clients algérois qui ont commandés ou non ?

II.2.2.3 Consultation avec sous-requêtes

Les conditions avec sous-requêtes permettent de comparer une expression ou une colonne au résultat d'une autre requête **Select** dite sous-requête.
La sous requête peut faire appel a une autre sous requête, etc.

Une condition de sous-requêtes peut être formulée selon l'une des possibilités suivantes :

- WHERE expression ***opérateur-de-comparaison***
  (SELECT...FROM...WHERE...condition…)

  Où ***opérateur-de-comparaison*** est: =,>, <, < > ou != (pour différent), <= , >=

- WHERE expression ***opérateur-de-comparaison*** **ALL/ANY**
  (SELECT...FROM...WHERE...condition…)

- WHERE expression **[NOT] IN**
  (SELECT.....FROM.....WHERE...condition...)

- WHERE expression **[NOT] EXISTS**
  (SELECT.....FROM.....WHERE...condition...)

Exemple 1:

```
SELECT   Nom,Marchandise
FROM     FOURNISSEUR
WHERE    Prix = (select max (Prix) from FOURNISSEUR) ;
```

Exemple 2 : On va reprendre une question précédente :

Question) Lister le nom et adresse des clients ayant commandé une marchandise fournit par 'BOUZIAD'

Exemple 3 :

Lister le(les) clients ayant le plus petit val-compte ? (utiliser un opérateur de comparaison avec les options AND et ANY) :
(*Rappelons par exemple :*

- *que « < = all » est un opérateur ensembliste qui signifie que la comparaison doit être vérifiée avec tous les éléments de l'ensemble*
- *et que « >= ANY » signifie que la comparaison doit être vérifiée avec au moins un élément de l'ensemble)*

<u>Exemple 4</u> : Fournisseur de pomme les moins cher ?

<u>Exemple 5</u> : Fournisseur achetant des produits qu'ils vendent ?

II.2.3 Consultation avec operateurs ensembliste

Les opérateurs ensemblistes sont : MINUX, INTERSECT et UNION

Syntaxe :

(Requête 1) {UNION/ INTERSECT/ MINUX} (Requête 2)

Exemple : Nom de toutes les personnes de la base de données ?

Attention : la liste des colonnes de « requête 1 » et « requête 2 » doivent être les mêmes (en nombre et en type).

II.2.4 Autre opérateurs

Un autre opérateur très utile est : EXISTS (et NOT EXISTS)

Syntaxe :

EXISTS (sous REQUETE)

Cette condition retourne vrai ou faux selon que la sous requête retourne un ensemble vide ou non.

Exemple : liste des fournisseurs qui sont clients ? (Utiliser l'operateur EXISTS)

Exercice : lister le nom et adresse du fournisseur et nom de marchandise les moins cher ?

II.2.5 Consultations ou Requête avec TRI

Pour trier les données (résultats) d'une requête par ordre croissant ou décroissant, il faut utiliser après les conditions ou après la liste de tables quand il n'y a pas de conditions, le terme 'ORDER BY' :

Syntaxe :

```
SELECT      …
FROM        …
[WHERE]
ORDER BY    colonne1     [ASC/DESC],
            colonne2     [ASC/DESC],
            …
            colonneN     ASC/DESC;
```

L'ordre se fait d'abord d'après la première colonne, puis la seconde, etc.....

Exemple : Lister tous les clients par ordre croissant sur l'adresse puis sur le nom.

Attention : Avec les chaines de caractères, la lettre minuscule "a" passe après la lettre majuscule "Z"

II.2.6 Consultation ou requête avec fonction

On peut utiliser deux types de fonction :

1. Arithmétique, (utilisé dans des expressions arithmétiques)
2. Agrégat.

II.2.6.1 Expressions Arithmétiques

- Peuvent être utilisée dans les clauses SELECT et WHERE
- Utilisent :
  - Colonnes des tables
  - Constantes
  - Operateurs arithmétiques : + , - , * , /
  - Fonctions arithmétiques (voir annexe)

Attention :

- les fonctions arithmétiques s'appliquent qu'aux colonnes de type 'Number '
- c'est des fonctions à un seul argument

Exemple 1 : Donner les prix des marchandises en $ (Dollars) sachant que le taux est de 100DA pour 1$

Exemple 2 : Donner le prix arrondis de toutes les marchandises en DA

II.2.6.2 Fonctions agrégat (ou de groupe) : (voir annexe pour la liste des fonctions)

- S'applique à un ensemble de valeurs d'une colonne d'une table
- Elles ont comme argument un nom de colonne ou une expression arithmétique
- Il faut utiliser le « DISTINCT » si on veut considérer que les valeurs différentes dans la fonction
- La fonction agrégat « COUNT » peut prendre comme argument le caractère « * ». Dans ce cas, elle rend comme résultat le nombre de lignes sélectionnées.

Exemple 1 : Nombre de client de la ville d'Alger

Exemple 2: Prix moyen des prix des marchandises

Attention : Il est interdit

- d'utiliser dans le SELECT une fonction d'agrégat avec d'autres colonnes à sélectionner
- interdit d'utiliser aussi une fonction agrégat dans la clause WHERE.

Exercice : Nombre de clients de la ville d'Alger dont le numéro de téléphone commence par 0661

II.2.7 Consultations (ou requêtes) avec groupement des résultats

II.2.7.1 Groupement sans condition

Avec les fonctions d'agrégat, SQL permet de grouper les lignes de données ayant des valeurs communes.

Syntaxe :

SELECT « *liste_ colonnes* » , « *liste_fct_agrégat* »
FROM …
[WHERE....]
GROUP BY « *liste_groupe_de_colonnes* » ;

où « *liste colonnes* » est un sous ensemble de la liste « *liste_groupe_de_colonnes* »

Exemple 1 : Quantité de chaque marchandise commandée ?

Rq : La fonction s'applique à un ensemble de valeurs d'une colonne.

II.2.7.2 Groupement avec condition

Pour introduire une condition sur les groupes, on utilisera la clause « HAVING »

Syntaxe :

SELECT « *liste_colonnes* »
FROM …
[WHERE …]
GROUP BY « *liste_groupe_de_colonnes* »
HAVING « *Conditions* » ;

Où « *Conditions* » s'applique à chaque groupe résultat

Exemple : Marchandise fournis par plus de deux fournisseurs

II.2.8 Modifications dans les tables

II.2.8.1 Insertion

Syntaxe :

INSERT INTO « nom_table » (col_i, col_j, …, col_n)
VALUES (valeur_i, valeur_j, …, valeur_n) ;

Exemple 1 :

```
INSERT INTO CLIENTS
VALUES  ('BOUZIAD', 'TIZI-OUZOU ', 0 )

INSERT  INTO CLIENTS (Nom, adresse)
VALUES ( 'BOUTEF' , 'Alger') ;
```

II.2.8.2 Suppression

Il s'agit de supprimer des lignes dans une tables et non de supprimer la table

Syntaxe :

DELETE FROM « nom_table «
[WHERE conditions] ;

Exemple 2 : Suppression des commandes de « BOUZIAD »

```
DELETE FROM cc
WHERE Nom= 'BOUZIAD' ;
```

Exemple 3 : Supprimer toutes les commandes de « BOUZIAD » du fournisseur « TOTO »

II.2.8.3 Modification

Le but de cette commande est de modifier le contenu d'une table et non sa structure !

Syntaxe :

```
UPDATE    «  nom_table »
SET       Col_i = Val_i,
          Col_j = Val_j ,
          ...
          Col_n = Val_n
WHERE     « Conditions » ;
```

Exemple : Augmenter de 10 % tous les prix des marchandises dont le prix est supérieur a 200 DA

III ANNEXE : LES FONCTIONS PREDEFINIES DANS ORACLE

Les fonctions prédéfinies de SQL sont :

1. les fonctions de conversion entre les différents types de données,
2. les fonctions d'agrégat,
3. les fonctions numériques,
4. les fonctions de chaînes de caractères,
5. les fonctions de date,
6. et des fonctions utiles pour l'environnement d'exploitation.

III.1 LES FONCTIONS DE CONVERSION

Les fonctions de conversion entre les différents types de données SQL les plus utilisées, sont les suivantes :

- ✓ **To_number([char[,fmt])** : convertit une donne de type CHAR ou VARCHAR2 en un nombre selon le format **fmt**
- ✓ **To_date(char[,fmt])** : convertit une chaîne de caractères en une valeur de type date selon le format de date fmt
- ✓ **To_char(date[,fmt])** : convertit une données de type date a une valeur de type VARCHAR2 selon le format spécifié fmt

III.2 LES FONCTIONS NUMERIQUES

Ces fonctions traitent les données de type NUMBER :

- ✓ **abs(n)** : donne la valeur absolue de n
- ✓ **ceil(n)** : retourne le plus petit entier supérieur ou égal à n
- ✓ **cos(n)** : retourne le cosinus de n, n étant exprimé en radian
- ✓ **exp(n)** : calcule la valeur exponentielle de n
- ✓ **floor(n)** : retourne la valeur entière de n
- ✓ **int(n)** : calcule le logarithme népérien de n
- ✓ **mod(n)** : retourne le reste de la division entière de m par n
- ✓ **power(n,m)** : retourne la valeur de n a la puissance m
- ✓ **round(n, [(m)])** : retourne la valeur de n arrondi à m positions à droite du point décimal

- ✓ **sign(n)** : retourne -1 si n est <0, 0 si n=0 , 1 si n>0
- ✓ **tan(n)** : retourne la tangente de n , n étant exprimé en radian
- ✓ **trunc(n):** retourne la valeur n tronquée a m positions décimales. Si m est négatif, la troncature se fait avant le point décimal.

III.3 LES FONCTIONS DE CHAINES DE CARACTERES

- ✓ **lower(char)** : met en majuscule ma chaine char
- ✓ **upper(char)** : met en minuscules la chaine char
- ✓ **substr(char,m,n)** : extrait de la chaine char une chaine de caractères de longueur n à partir de m ieme caractère , le m ieme caractère étant compris dans la chaine extraite
- ✓ **lenght(char)** : donne la longueur de la chaine char en caractères.

III.4 LES FONCTIONS DE DATE

- ✓ **add_months(d,n)** : ajoute n mois à la date d. Le résultat est une date
- ✓ **last_day(d)** : retourne la date du dernier jour du mois de la date d
- ✓ **months_between(d1,d2)** : retourne le nombre de mois entre les dates d1 et d2.
- ✓ **next_day(d,char)** : retourne la date du premier jour de la semaine de la date d nomme par char .
- ✓ **round(d[,format])** : retourne une date arrondie selon le format spécifie (DD ,MM ou YY).
- ✓ **sysdate** : retourne la date et l'heure courante du système.
- ✓ **trunc(d,[format])** : retourne la date d tronquée selon le format spécifie. Si le format est DD, la date est tronquée au premier jour de la semaine. Si le format est MM, la date est tronquée au premier jour du mois. Si le format est YY, la date est tronquée au premier jour de l'année.

III.5 LES FONCTIONS D'AGREGAT

Oracle offre entre autres les fonctions de base pour déterminer des calculs statistiques ; ces fonctions ne sont utilisables que dans la *liste_de_colonnes* d'une requête et dans le HAVING de la clause WHERE.

- ✓ **count(*)** : Calcule le nombre de lignes satisfaisant la requête, y compris les lignes nulls

- ✓ **count([DISTINCT/ALL]colonne)** :
  Calcule le nombre de valeurs non null de la colonne "colonne" satisfaisant la requête ; avec l'option ALL qui est l'option par défaut, on compte toutes les valeurs qui satisfont la requête

- ✓ **sum([DISTINCT/ALL]colonne)** :
  Calcule la somme des éléments de la colonne de type NUMBER qui satisfont la requête ; la colonne peut être le produit de plusieurs colonnes de type NUMBER

Exemple :

select sum(prixunit) from articles ;

III.6 FONCTIONS STATISTIQUES

- ✓ **avg([DISTINCT/ALL] colonne)** : fonctionne comme SUM, mais calcule la moyenne.

- ✓ **max({DISTINCT/ALL]colonne)** : calcule le max des éléments de la colonne

- ✓ **min([DISTINCT/ALL] colonne)** : calcule le minimum

- ✓ **stddev([DISTINCT/ALL]colonne)**: calcule l'écart type

- ✓ **variance([DISTINCT/ALL]colonne)** : calcule la variance

III.7 AUTRES FONCTIONS

- NVL :
  nvl (exp1, exp2) : retourne exp2 si exp1 est nulle. Sinon , retourne exp1.

- USER :
  user : retourne le nom de l'utilisateur courant.

- DECODE **:**
  Decode (expression/colonne, recherche1, resultat1 [,recherche2,resultat2]...[defaut]) :
  Compare (comparaison de type égal =) une expression ou une colonne a une condition de recherche explicitée et renvoie le résultat qui dépend de cette condition (c'est à dire le résultat qui est dans la liste, juste après la condition de recherche qui est égale à l'expression ou à la colonne) .

  Si l'expression est égale a aucune des conditions de recherche, l'utilisateur peut donner comme résultat un résultat par défaut.

Exemple :

decode(stock , 0 , 'Stock épuise ' , 10 , 'Alerte épuisement du stock', ‘bon niveau de stock') :

Si la valeur stock est égale à 0, le résultat retourné est 'Stock épuisé’, si elle est égale à 10, le résultat est 'Alerte épuisement du stock', si elle ni égale à ni égale à0, le résultat est 'bon niveau de stock'

III.8 LES SEQUENCES

Une séquence est utilisée pour assurer une numérotation séquentielle d'une colonne particulière.

III.8.1 CREATION

Syntaxe

CREATE SEQUENCE nom_sequence [INCREMENT BY int1] [START WITH int2] [MAXVALUE int3][MINVALUE int4][CYCLE/NOCYCLE] ;

Où int1,..int4 sont des entiers positifs ou négatifs.

- Increment by int1 : la séquence s'incrémente tous les int1 entiers
- Start with int2 : la séquence commence à l'entier int2. int2 est égal à 1 par défaut
- Maxvalue int3 : int3 est la limite supérieure de la séquence
- Minvalue int4: int4 est la limite inférieure de la séquence
- Cycle : lorsque la séquence atteint sa limite supérieure ou inférieure, elle recommence à l'entier int2
- No cycle : la séquence ne peut être utilisée si elle atteint une de ses limites. NO CYCLE est l'option par défaut

L'incrémentation d'une séquence se fait par la commande NEXTUAL et la consultation de la valeur se fait par CURRVAL.

Exemple :

- ✓ create sequence s1 increment by2 start with 100 maxvalue 1000 cycle ;

va générer les entiers : 100, 102, 104.....1000,100,102,104,..etc

- ✓ create sequence s2 increment by -1 ;

va générer 1,0,-1,-2,....-infini

III.8.2 SUPPRESSION

Suppression de la séquence **nom_sequence** :

Drop sequence nom_sequence ;

III.9 DICTIONNAIRE DES DONNEES

Ce dictionnaire peut être consulté en lecture par l'administrateur et aussi par les utilisateurs.

On distingue 3 types de dictionnaires ou 3 types de vues sur le DICTIONNAIRE.

III.9.1 VUES RELATIVES AUX OBJETS D'UN UTILISATEUR

Les noms de ces vues commencent toujours par le préfixe : "USER_"

Les vues les plus utilisées sont :

- ✓ USER_CATALOG ou CAT : liste des tables, vues, synonymes et séquences appartenant à l'utilisateur (nom et type d'objet : table_NAME, table_TYPE...)
- ✓ USER_CONSTRAINTS : liste des contraintes définies sur les tables de l'utilisateur.
- ✓ USER_COL_COMMENTS : liste des commentaires définies sur les tables de l'utilisateur (nom de table, nom de colonne, commentaires)
- ✓ USER_INDEXES ou IND : liste des indexes crées par l'utilisateur.
- ✓ USER_OBJECTS ou OBJ : liste des objets appartenant à l'utilisateur (nom, identifiant, type, date de création, date de dernière modification de l'objet).
- ✓ USER_TRIGGERS : pour les triggers.
- ✓ USER_SOURCE : code des procédures, fonction et packages.
- ✓ DUAL : contient par exemple la date courante : select sysdate from dual ;

III.9.2 VUES RELATIVES AUX OBJETS ACCESSIBLES A UN UTILISATEUR

- ✓ ALL_nom_du_dic

Exemple : select * from all_catalog,

III.9.3 VUES RELATIVES AUX ADMINISTRATEURS

- ✓ DBA_nom_du_dic

Exemple :

- DBA_COL_COMMENTS
- DBA_CONSTRAINTS.